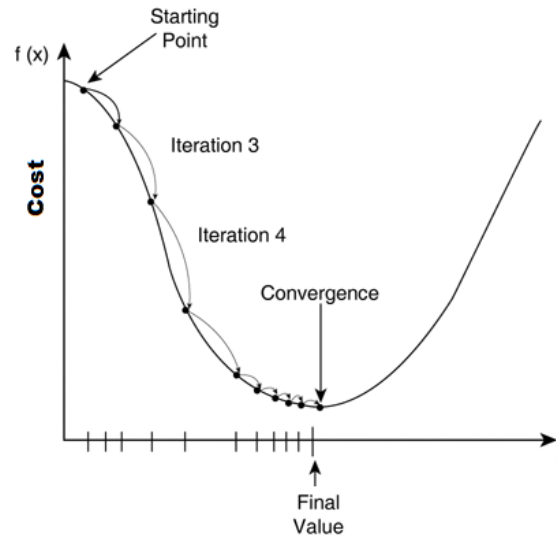


DEEP LEARNING

Empirical Risk Minimization

Dr. Kuppusamy P



Optimization

Optimization

- The process of estimating the output that maximizes its performance (accuracy, precision, recall) and decreases the error (loss / cost).
- In optimization, model learns from the known (training) data .

Need of optimization

- Model finds the maximum value (accuracy, precision, etc.,) will be the best solution to given problem.
- In Deep Learning, model mostly care about generalization i.e., the unknown (test) data.

Optimizer

- An algorithm or function that **adapts** the neural network's attributes such as **learning rate** and **weights**.
- It updates the attributes to improve the accuracy, and reduces the total loss.

Pure Optimization and Learning

Pure optimization

- Optimizing cost $J(\theta)$ to minimize the error.
- If we have true data, it becomes an optimization problem.
- E.g., Compute the **height** of the Indian people
 - **True data** – Nation wide population. But it is impossible to approach entire population.
 - **Training data** – Set of people considered across India. It provides approximate solution to this problem.

Learning

- The acquisition of knowledge through examining patterns from experience (data).

True Risk & Empirical Risk

Expected Generalization Error (True Risk)

- The expected generalization error (risk) is calculated over the **true data** distribution.
- Pure optimization is often just concerned with **optimizing cost $J(\theta)$ directly** based on **True data**.

Empirical Risk (Expected Loss)

- In real-time application, we are **unknown** exactly how well an algorithm will work (the true "risk") because we don't know the **true** data distribution.
- Instead we can **measure** model's performance on a **known** set of **training data** (the "empirical" risk).
- The most straightforward function is the **empirical risk** based on **training data**.

How **Learning** differs from **Pure Optimization**?

- Machine learning usually acts **indirectly** for optimization i.e., uses training dataset.
- Most of the machine learning algorithms use some performance measure P , that is defined w.r.t the test set, and may also be intractable (not easily managed or controlled).
- Improve the metric P by **reducing** the error using cost function $J(\theta)$.
- It is achieved by updating the parameters during optimization.
- But it is in **contrast** to pure optimization.
- **Pure optimization** goal is **minimizing $J(\theta)$ directly** that based on **true dataset**.
 - E.g., Compute the **height** of the Indian people
 - **True data** – Nation wide population.
 - True risk – Loss calculated from True Data (distribution of the data is **not known** priorly)
 - **Training data** – Set of people considered across India.
 - **Empirical Risk** – Loss calculated from **Training data**.

Law of Large numbers:

- Empirical **mean** of m independent identical distribution of **random variables converges** to their (identical) mean for growing sample size m .

Objective function w.r.t Data Generalization Error (True Risk)

- **Computing loss** for all possibilities is considered to be the “**true risk**”.
- Drawback: The distribution of the data is **not known** priorly.
- Cost **Objective function** w.r.t **True Data** is

$$J(\theta) = E_{(x,y) \sim P_{data}}(L(f(x; \theta), y))$$

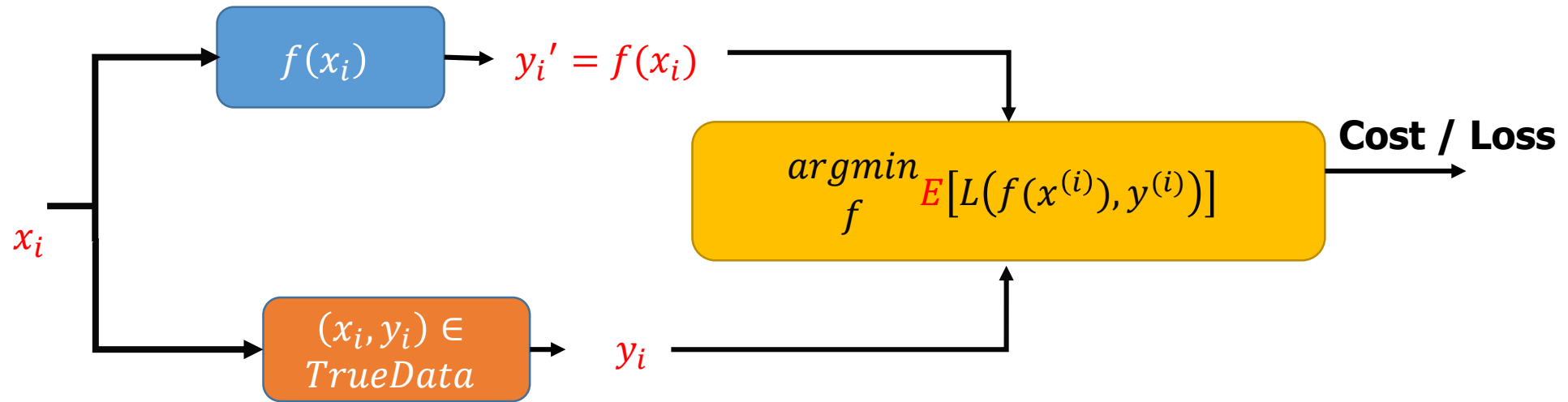
- L – per-example loss function, $f(x)$ - predicted output for the input x .
- **Minimize** the corresponding objective function where expectation is across the **data generating distribution** (P_{data}) **rather than** over **finite training** ($\hat{P}_{data}$) **set** as follows:

$$J(\theta) * = \underset{f}{argmin} E_{(x,y)}(L(f(x), y))$$

- This **Expectation loss** or **Risk** can be **approximated** with **finite** training data.
- The goal of a **machine learning algorithm** is **to reduce** this expected generalization error or risk .
- So, **convert** a machine learning problem into an **optimization** problem to **minimize** the **Expected loss (E)** on the **training set** i.e., **replacing** the **true** distribution $p(x, y)$ with the **empirical** distribution $\hat{p}(x, y)$ defined by the training set.

True (Statistical) Risk

- The parameterization is a logic that allows the function to predict the testing samples.
- **Statistical** risk minimization learns from model.
- The components of True risk are True data, Function, and loss function.



Cost function

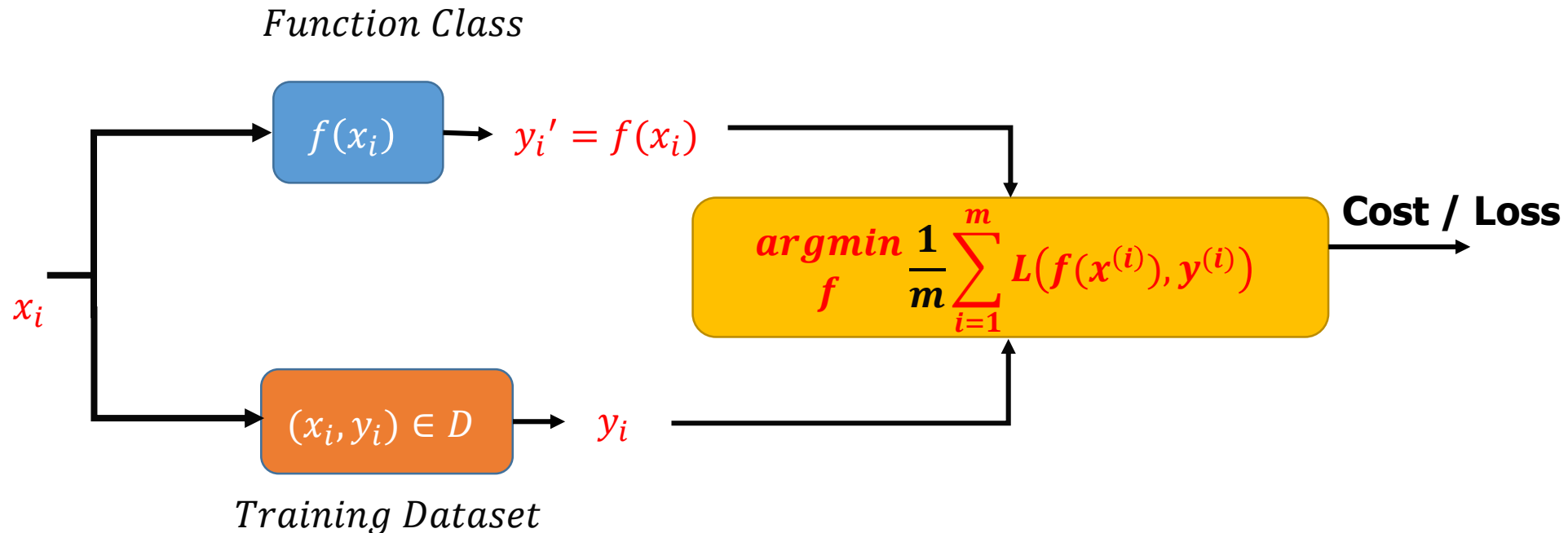
- **Replace** the **general (statistical) risk** minimization with **empirical** minimization

$$J_{\textcolor{red}{s}} = \underset{f}{\operatorname{argmin}} E_{(x,y)}(L(f(x), y)) \Rightarrow J_{\textcolor{red}{E}} = \underset{f}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L(f(x^{(i)}), y^{(i)})$$

- Since objective functions are close, then optimal also will be close $J_s \approx J_E$.
- But **Statistical** minimization **need not** to be **close** to empirical minimization.
- The ERM solutions is trivial.
- It makes $f(x) = y$, for all samples in the training dataset.
- As trivial as nonsensical. It provides **no information** about **test samples**.

Learning Parameterization

- The parameterization is a logic that allows the function to predict the testing samples.
- Parameterized risk minimization learns from training dataset.
- Statistical Risk is **replaced** by the empirical risk.
- The function class determines generalization from **training set** to unseen (test) samples.



Empirical risk Limitations

- But empirical risk minimization is :
 1. **Prone to overfitting**: It's occur due to models simply memorize training set.
 2. The most effective modern gradient descent based optimization algorithms **not able to use derivatives** (i.e., 0-1 loss function have the derivative either zero or undefined everywhere).
- These three challenges lead **rare use** of **empirical risk minimization**.
- So, apply slightly different approach based on **Quantity**. Model must optimize the cost function even more different from expected accuracy, and cost.

0-1 Loss function

- To train the model, applying the Mean Squared Error loss functions and most frequently.
- But MSE may not reflect the true accuracy of the classifier.
- So apply 0-1 loss function which gives error as **1** if it is **misclassified**, and 0 if the prediction is correct . It counts the number of misclassified items
- 0-1 loss function is an important metric for the quality of **binary** and **multiclass** classification algorithms. It is a very basic loss function.
- Assume a classification problem with data set $\{(x_1, y_1), (x_2, y_2), \dots (x_n, y_n)\}$, x_n are features and y_n are true labels.
- Given a hypothesis function $h(x)$, the loss function $L: (h(x_n), y_n) \rightarrow \mathbb{R}$ takes the prediction (i.e., $h(x_n)$), and true label for particular input and returns a penalty.
- Find a hypothesis such that it minimizes the empirical risk (i.e., minimizes the misclassifications):

$$\mathbb{R}_l(h) = E_{emprical}(L(h(x), y)) = \frac{1}{m} \sum_{i=1}^m L(h(x^{(i)}), y^{(i)})$$

0-1 Loss function

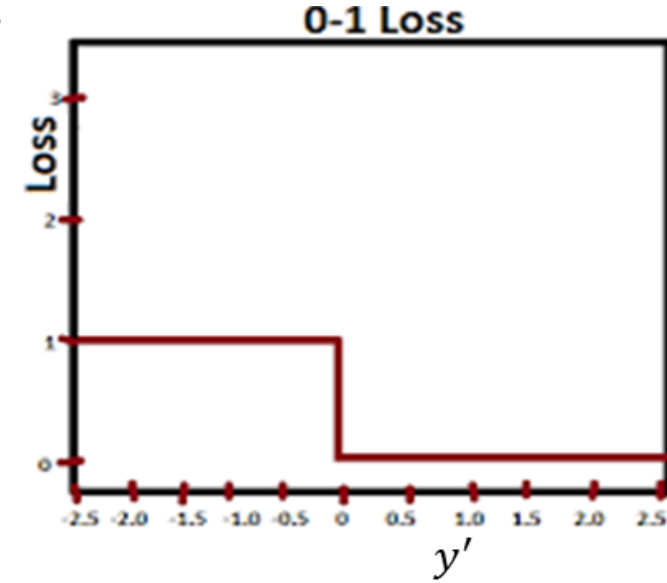
- In binary classification, a common loss function that is used is the 0-1 loss function:

$$\delta_{y'_i \neq y_i} = \begin{cases} 0 & y == y' \\ 1 & y \neq y' \end{cases}$$

- In general, the loss function that we use **cannot** be optimized efficiently.

E.g., 0-1 loss function is **discontinuous** and **not differentiable**.

- So, consider another loss function that will **reduce** the cost, which is called the **surrogate loss** function.
- 0-1 loss leads to estimate the **mode** (count of misclassified samples) of the target distribution (L_1 loss for estimating **median** and L_2 loss for estimating **mean**).



$$L_{01} = \frac{1}{m} \sum_{i=1}^m \delta_{y'_i \neq y_i}$$

0-1 Loss function

- It is non-smooth and non-convex
- Small changes in w and b leads to huge loss.
- Loss will increase drastically with **small changes** in the model.

Limitation:

- 0-1 Loss function is not differentiable and non-continuous. Therefore it is **not possible** to apply **gradient descent** methods. 0-1 loss function makes this optimization hard if the data is not linearly separable.

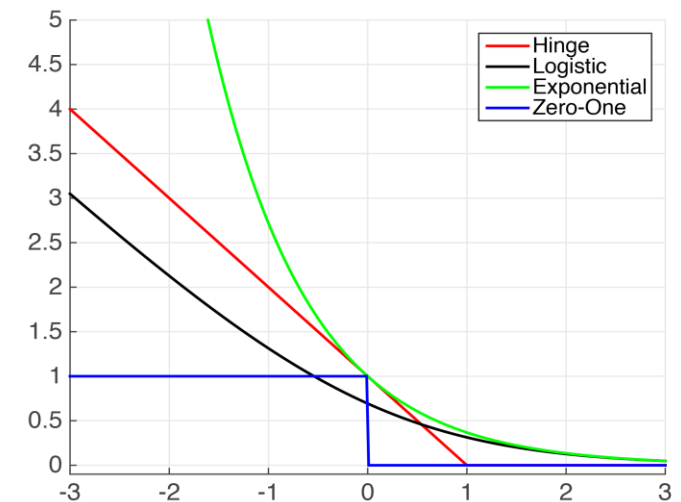
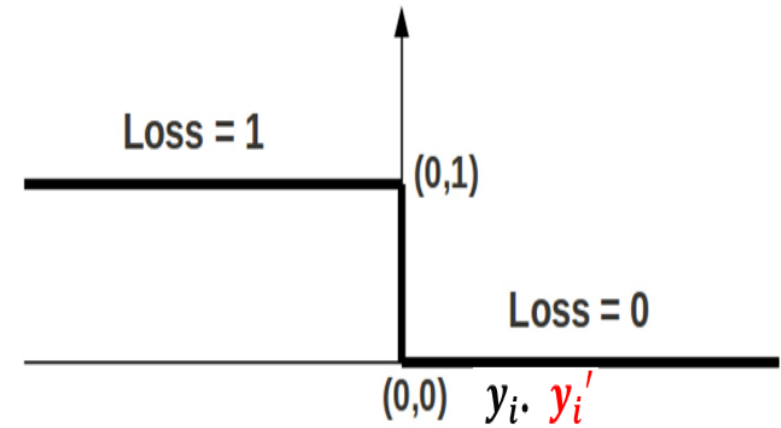
Need of Surrogate Loss Function for Classification Problem

- So, search a surrogate (alternate) loss function as a cost function that is differentiable, and convex.
- Obviously, surrogate loss function actually provides good results by learning more patterns, and classifier achieves optimal risk i.e., highest accuracy and decreases loss.

- Loss function applied for classification actually **should be optimized** efficiently.

E.g.,

- Exactly minimizing expected 0-1 loss is typically intractable (exponential in the input dimension), even for a **linear** classifier.
- Approximating 0-1 loss with a convex function is known as **surrogate loss**.



Surrogate Loss Functions for Classification Problems

- **Zero-one loss** is 0 when the prediction is correct, and 1 when incorrect.
- **Hinge-loss** decreases, improves the training error. If $y * y'$ is **+ve** value, then hinge loss is **0**. If it is **-ve** value, then hinge loss is **huge**.

$$\text{Hinge Loss} = \max(0, 1 - y * y')$$

E.g.,

- Surrogate loss function could be hinge loss (for SVM) for 0-1 loss function.
- When **Hinge-loss** decreases, the training error is decreasing.
- If $y * y'$ is **+ve** value, then hinge loss is **0**. If it is **-ve** value, then hinge loss is **huge**.
- Hinge loss is convex and easy to optimize using conventional optimisation methods.
- This function acts as a proxy (Surrogate) for the actual loss that need to minimize.
- **Log loss** (**Binary, Categorical** cross entropy) is the most popular loss function, since its outputs are very well-tuned. $L = -y_i \cdot \log(y'_i)$
- Model can effectively learn the underlying patterns and correlations necessary for accurate classification.

Surrogate Loss Functions

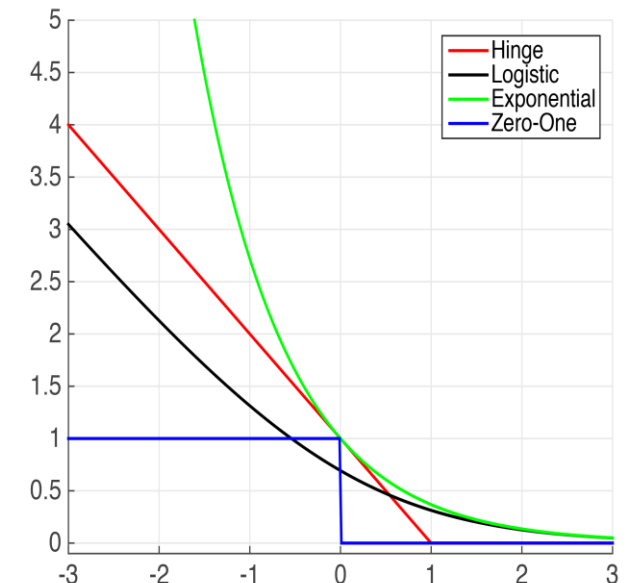
- The **Negative Log-Likelihood (NLL)** is a commonly used **surrogate** (substitute) for the 0-1 loss in machine learning.
- It allows the model to estimate the conditional probability of different classes given an input and is **closely related** to the concept of maximum likelihood estimation.

E.g.,

- Assume the dataset with spam and ham emails. Classify an email as spam or ham.

After training, model outputs the following probabilities for the **two**

- Probability of "spam": 0.8
- Probability of "ham": 0.2
- After applying NLL, NLL on spam is $-\log(0.8) = -(-0.223) = 0.223$
- NLL on ham is $-\log(0.2) = -(-1.609) = 1.609$
- Loss is **low** for Spam. It shows that model works better.



MLE

- MLE is a statistical principle to find the model parameters that maximize the likelihood of the observed data.
- In this example, the likelihood is the probability of observing the true label given the input.
- The model parameters are adjusted during training to **maximize** the likelihood of the training data. *Probability of "spam": 0.8 & Probability of "ham": 0.2*
- By **minimizing** the NLL loss (or equivalently, maximizing the log-likelihood), we are effectively finding the model that is most likely to have generated the observed data.

$$NLL \text{ on spam} = -\log(0.8) = 0.223 \text{ \& } NLL \text{ on ham} = -\log(0.2) = 1.609$$

- if the model **accurately** estimate the conditional probability of "spam" given the input, it will assign a **higher** probability to "spam" than "ham."
- As a result, the **NLL** loss will be **lower**, indicating a better fit between the predicted probabilities and the true label.

Optimization for training algorithms

- Training algorithms **do not** usually **halt** at a **local minimum**.
- Instead, a machine learning algorithm usually minimizes a surrogate loss function but **halts** when a **convergence** criterion based on **early stopping**.
- Optimization encompasses finding the **best solution** across various domains.
- Optimization specifically focuses on finding the optimal parameters for a machine learning algorithms or statistical model to minimize a chosen loss function.
- The process involves **iterative** updates of the model's parameters using optimization algorithms tailored for machine learning tasks.

Comparison of two hypothetical estimates of the gradient

- Assume two datasets with 100 samples and 10,000 examples.
- The latter requires 100 times more computation than the former, but reduces the standard error of the mean only by a factor of 10.
- Most optimization algorithms **converge** much **faster** based on total computation, not in terms of number of updates, if rapidly compute **approximate** of the gradient rather than slowly computing the **exact** gradient.

Challenges in Neural Network Optimization

- Traditionally, machine learning has avoided the difficulty of general optimization by carefully designing the objective function and constraints to ensure that the optimization problem is convex.
- When training neural networks, model challenge the general non-convex case (neural networks with multiple hidden layers)
- There are several prominent challenges involved in optimization for training deep models.
 - Local minima
 - Plateaus, Saddle Points and Other Flat Regions
 - Cliffs and Exploding Gradients
 - Inexact Gradients
 - Long term dependencies
 - Poor Correspondence between Local and Global Structure

References

- <https://stats.stackexchange.com/questions/263712/what-is-a-surrogate-loss-function>

Empirical risk minimization

- When build the learning model, need to choice the function that minimizes the empirical risk i.e., the difference (delta) between the predicted output, and the actual output for the data points in the dataset.
- This process of finding this function is called empirical risk minimization.
- E.g.,

	height	width	y=exploded?	Model 1 prediction $\hat{y}_1$
chip 1	0.8	0.8	1	1
chip 2	0.3	0.25	0	0
chip 3	0.2	0.8	0	1
chip 4	0.3	0.7	0	1
chip 5	0.9	0.7	1	1

$$L_{\mathcal{S}}(h) = \frac{1}{n} \sum_{i=1}^n \ell_{01}(h(\mathbf{x}_i), y_i) = \frac{1}{5}(0 + 0 + 1 + 1 + 1 + 0) = \frac{1}{5}3 = \frac{3}{5}.$$